Neue Themen für die 808

12.02.2025

Plan für die Woche

Montag

- StringBuilder

Dienstag

- DateTime API

Mittwoch

- Stack und Heap
- Garbage Collector

Donnerstag

- To be planned

Freitag

- To be planned

Plan für heute

- Wiederholung StringBuilder
- Strings vs. StringBuilder

StringBuilder

Wie viele Objekte resultieren von diesem Codeabschnitt? Wie viele landen direkt im Garbage Collector?

```
String s = "";
for (char current = 'a'; current <= 'z'; current++){
    s += current;
}
System.out.println(s);
```

A. Maximal 1
B. Maximal 20
C. Maximal 25
D. Maximal 27

Grundlagen

- vorheriges Beispiel zeigt, dass Strings ineffizient werden können!
  - Jede Änderung erzeugt ein neues Objekt im Speicher, da String immutable (=unveränderbar) sind
  - Viele Änderungen führen zu Leistungsproblemen
- Lösung: StringBuilder
  - Ist mutable (=änderbar)
  - Änderungen werden am selben Objekt vorgenommen
  - Spart Speicher und verbessert die Performance

Initialisierung

- Ein StringBuilder-Objekt mit einer initialen Zeichenkette „Hallo“

```
StringBuilder b = new StringBuilder("Hallo");
```

- Erstellt einen leeren StringBuilder
- Standardkapazität liegt bei 16, wird automatisch erweitert

```
StringBuilder c = new StringBuilder();
```

- Standardkapazität wird auf 5 gesetzt, wird automatisch erweitert

```
StringBuilder d = new StringBuilder(5);
```

Methoden

- **append(String s):**
  - **Erweitert das StringBuilder-Objekt um einen String s**

```
public static void main(String[] args) {
    StringBuilder b = new StringBuilder("Hallo");
    b.append(" mein Name ist Anita");
    System.out.println(b);

    //Ausgabe: Hallo mein Name ist Anita
}
```

Methoden

- length()
  - Gibt die Länge der Characters im StringBuilder zurück

```
public static void main(String[] args) {
    StringBuilder b = new StringBuilder("Hallo");
    b.append(" mein Name ist Anita");
    System.out.println(b.length());

    //Ausgabe: 25
}
```

Methoden

- insert(int index, String str)
  - Fügt an die Index-Position den String str ein

```
public static void main(String[] args) {
    StringBuilder b = new StringBuilder("Hallo");
    b.append(" mein Name ist Anita");
    System.out.println(b.insert(19, „ immernoch"));

    //Ausgabe: Hallo mein Name ist immernoch Anita
}
```

Methoden

- delete(int start, int end)
  - Löscht von start bis end-1 die Character
  - Wenn end-index nicht existiert, wird es trotzdem ausgeführt

```
public static void main(String[] args) {
    StringBuilder b = new StringBuilder("Hallo");
    b.append(" mein Name ist Anita");
    System.out.println(b.delete(20, 25));

    //Ausgabe: Hallo mein Name ist
}
```

- deleteCharAt(int index)
  - Löscht den Character an Position des indexes

```
public static void main(String[] args) {
    StringBuilder b = new StringBuilder("Hallo");
    b.append(" mein Name ist Anita");
    System.out.println(b.deleteCharAt(20));

    //Ausgabe: Hallo mein Name ist nita
}
```

Methoden

- reverse():
  - Gibt die Character im StringBuilder von hinten nach vorne aus

```
public static void main(String[] args) {
    StringBuilder b = new StringBuilder("Hallo");
    System.out.println(b.reverse());

    //Ausgabe: ollaH
}
```

Methoden

- toString():
  - Konvertiert den StringBuilder in einen String

```
public static void main(String[] args) {
    StringBuilder b = new StringBuilder("Hallo");
    System.out.println(b.reverse().toString());

    //Ausgabe: ollaH
}
```

Mutability

- es wird an einem Objekt gearbeitet

```
public static void main(String[] args) {
    StringBuilder b = new StringBuilder("Hallo");
    b.append(" mein Name ist");
    StringBuilder c = b.append(" Anita");
    System.out.println(c);

    //Ausgabe: Hallo mein Name ist Anita
}
```

Method Chaining

- auch hier ist Methodenverkettung möglich

```
StringBuilder b = new StringBuilder("Hallo");
StringBuilder c = b.append(" mein Name ist Anita").append(" !").reverse().deleteCharAt(10);
```

Was ist die Ausgabe?

```
StringBuilder b = new StringBuilder("Hallo");
StringBuilder c = b.append(" mein Name ist");
b = b.append(" Anita").append(" !");
System.out.println(b);
System.out.println(c);
```

A. Hallo mein Name ist Anita !
   Hallo mein Name ist Anita !

B. Hallo Anita !
   Hallo mein Name ist

C. Hallo mein Name ist Anita !
   Hallo mein Name ist

Aufgabe ca. 30 min

Du hast einen String und musst diesen so umwandeln, dass er in ein bestimmtes Zielformat passt. Du sollst dabei die Methoden der Klasse StringBuilder verwenden, um den String zu manipulieren.

String input = "ssapsthcamdnullotttsinereimmargorp";

1. Dreht den String um. Was steht da?

2. Formattiert den String so um, dass folgender String rauskommt:

   Output : Programmieren ist toll und macht Spaß!

Achtet auf die Indexe! Wenn ihr etwas hinzufügt/wegnehmt, passen sich die Indexe mit an!

String vs. StringBuilder

Merkmal	String	StringBuilder
Mutabilität	Unveränderlich (immutable)	Veränderlich (mutable)
Speicherverbrauch	Höher	Niedriger
Leistung bei Änderungen	Langsam	Schnell
Verwendung	Konstanten, unveränderliche Daten	Häufige Änderungen

Equality

== im Kontext von StringBuildern

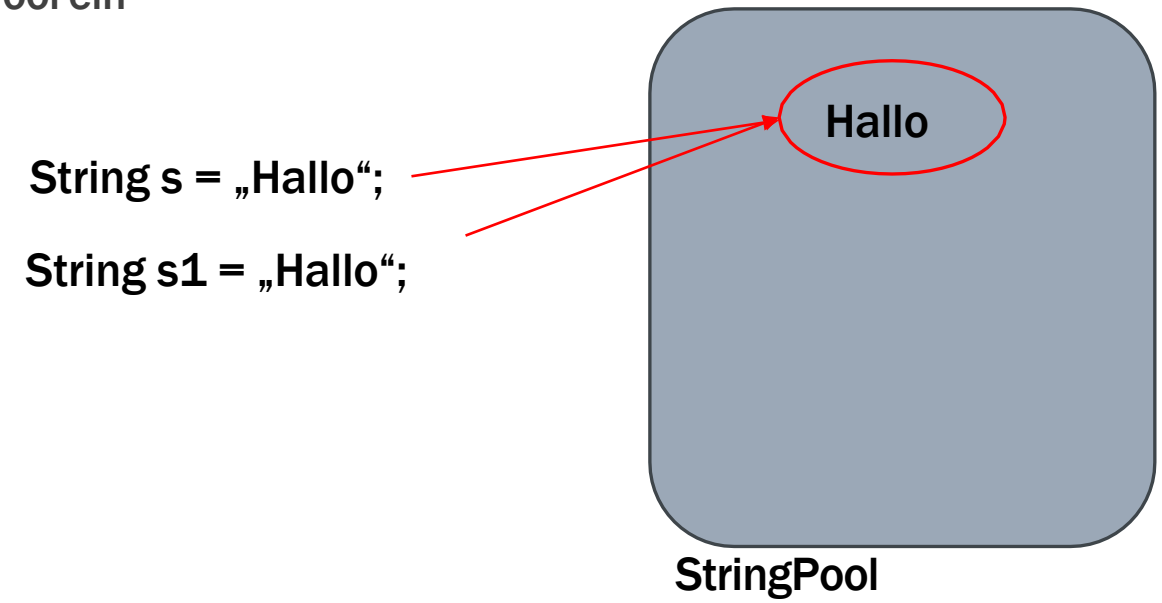
```
StringBuilder one = new StringBuilder();
StringBuilder thesecondone = new StringBuilder();
StringBuilder three = one.append("Hallo");
System.out.println(one == thesecondone);          //false
System.out.println(one == three);                 /true
```

== im Kontext von Strings

```
String s = "Hallo";
String s2 = "Hallo";
System.out.println(s == s2);            //true
```

StringPool

- ist eine Speicherstelle, in welcher Java String-Literale abspeichert
- bevor ein String-Literal in den StringPool landet, checkt Java, ob es nicht bereits existiert
  - Wenn es existiert, wird die Referenz auf das Bestehende Literal zurückgegeben
  - Wenn nicht, fügt Java das Literal in den StringPool ein



== im Kontext von Strings

```
String s = "Hallo";
String s2 = "Hallo  ".trim();
System.out.println(s == s2);                //false
```

Immutable

- sobald ein String-Objekt kreiert wurde, kann es nicht verändert werden
- man muss demnach einen neuen String erstellen
  - Bzw. den Pointer auf ein neues String-Objekt verweisen lassen
- die Anwendung von Methoden auf einen String resultieren in einer Erzeugung eines String-Objekts auf dem Heap

```
public class ImmutableStringExample {
    public static void main(String[] args) {
        String original = "Hallo";
        original.concat("Ola");

        System.out.println("Original: " + original);
       //Ausgabe : Hallo
    }
}
```

```
public class ImmutableStringExample {
    public static void main(String[] args) {
        String original = "Hallo";
        String modified = original.concat(" Welt!");

        System.out.println("Modifiziert: " + modified);
        //Ausgabe: Hallo Welt!
    }
}
```

== im Kontext von Strings

```
String s = new String("Hallo");
String s2 = "Hallo";
System.out.println(s == s2);                //false
```

.equals()

- vergleicht IMMER nur den Inhalt der Strings

```
String s = new String("Hallo");
String s2 = "Hallo";
System.out.println(s.equals(s2));                    //true
```

- StringBuilder hat equals() nicht überschrieben -> vergleicht also immer noch Referenzwerte (==)

== vs. .equals()

Vergleich	String	StringBuilder
==	Vergleicht Referenzen (Speicheradressen)	Vergleicht Referenzen (Speicheradressen)
.equals()	Vergleicht den Inhalt der Strings	Hat keine .equals()-Methode überschrieben

Aufgabe (ca. 5 min)

```
String str1 = "Hallo Welt";

String str2 = "Hallo Welt";

String str3 = new String("Hallo Welt");
```

Vergleiche die folgenden String-Objekte mit == und .equals() und gib jeweils das Ergebnis (true/false) aus:

```
str1 == str2
str1.equals(str2)
str1 == str3
str1.equals(str3)
```

Überprüfe dein Ergebnis in der IDE.

Aufgabe (ca. 5 min)

```
StringBuilder sb1 = new StringBuilder("Hallo Welt");

StringBuilder sb2 = new StringBuilder("Hallo Welt");
```

Vergleiche die folgenden StringBuilder-Objekte mit == und .equals() und gib das Ergebnis (true/false) aus:

```
sb1 == sb2
sb1.equals(sb2)
sb1.toString().equals(sb2.toString())
```

Überprüfe dein Ergebnis in der IDE.